

Input Data

Identification numbers (in given order):

A102, A25, A7, B100, B12, A120, B3, A45

OUTPUT

Inorder Traversal:

A102 A120 A25 A45 A7 B100 B12 B3

Search Comparison:

A120 : BST = 3 comparisons, Linear = 6 comparisons

B3 : BST = 6 comparisons, Linear = 7 comparisons

A45 : BST = 4 comparisons, Linear = 8 comparisons

B100 : BST = 4 comparisons, Linear = 4 comparisons

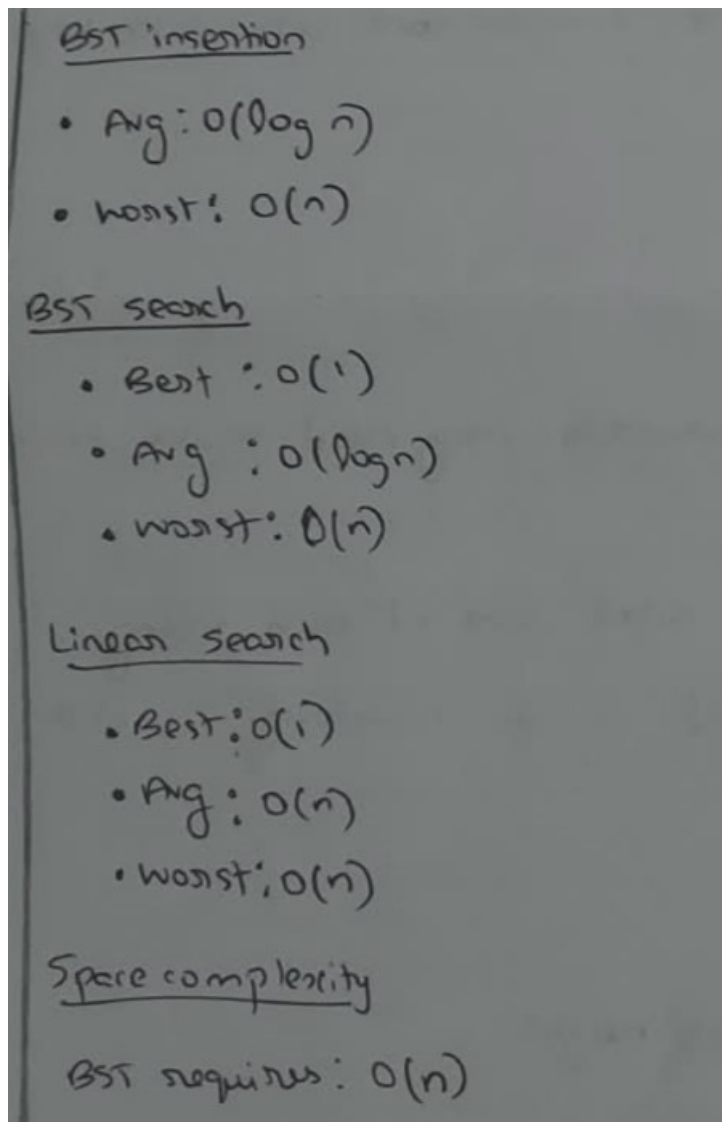
COMPARISON TABLE

Feature	BST Search	Linear Search
Data Structure	Binary search tree	Array/List
Best case	$O(1)$	$O(1)$
Avg case	$O(\log n)$	$O(n)$
Worst case	$O(n)$	$O(n)$
Search for A120	4	6
Search for B3	3	7
Search for A45	2	8
Search for B100	2	4
Suitable for growing database	Yes, if reasonably balanced	Less suitable

TRACE TABLE

Search key	BST path	BST comparisons	Linear-search comparisons
A120	A102 → B100 → B12 → A120	4	6
B3	A102 → B100 → B3	3	7
A45	A102 → A25 → A45	3	8
B100	A102 → B100	2	4

COMPLEXITY ANALYSIS



CONCLUSION

For the given government database, the BST is more suitable than Linear Search when frequent searches are required. In the execution, BST required fewer comparisons for all the selected keys.

The BST provides approximately $O(\log n)$ search performance when the tree remains balanced, whereas Linear Search requires $O(n)$ time in the average and worst cases. However, an unsuitable insertion order can make the BST skewed and reduce its performance to $O(n)$.

Therefore, as the database grows, a balanced BST such as an AVL Tree or Red-Black Tree would be a better choice for maintaining efficient searches